

VNUHCM – UNIVERSITY OF SCIENCE
FACULTY OF DATA SCIENCE

ACADEMIC COURSE REPORT
**KNOWLEDGE
REPRESENTATION**

**HLG LLM UTILS: AN ADVANCED GRAPH RAG
FRAMEWORK WITH MULTI-AGENT
ORCHESTRATION, HIERARCHICAL LEIDEN
COMMUNITY DETECTION, AND CYPHER QUERY
SELF-CORRECTION**

Instructor: Assoc. Prof. Dr. Do Van Nhon
Students: 1. Nguyen Hoai Nam – Student ID: 25C01014
2. Pham Cung Le Nhan Vu – Student ID: 25C01049
Major: Data Science
Academic Intake: K35

Executive Abstract

Standard Retrieval-Augmented Generation (Naive RAG) systems based on flat chunking and vector space similarity (e.g., Cosine Similarity over high-dimensional embeddings) suffer from two fundamental bottlenecks: (1) **Global Context Fragmentation**, causing failures in answering corpus-wide macro-summarization queries; and (2) **Multi-Hop Reasoning Failures**, which prevent LLMs from traversing complex dependency chains across disparate documents. This report presents **HLG LLM Utils**, a modular, production-grade Graph RAG framework built on a Python monorepo architecture (`uv workspaces`).

Our framework introduces key contributions in Knowledge Representation and Information Retrieval:

1. **Formal Property Knowledge Graph Modeling:** Automated extraction of entities ($\mathcal{E}$), relationships ($\mathcal{R}$), and claims/covariates from unstructured text, persisting them onto a Neo4j Property Knowledge Graph. Supports **Gleaning** (iterative extraction refinement) to maximize knowledge coverage.
2. **Hierarchical Leiden Community Detection:** Partitioning the entity graph into multi-level community hierarchies, combined with parallel Map-Reduce LLM summarization for Global Search queries.
3. **Cypher Query Self-Correction State Machine:** A LangGraph state machine translating natural language queries into executable Cypher statements, featuring a 3-iteration error feedback loop against Neo4j execution logs.
4. **ARK Autonomous Multi-Agent Exploration:** Implementing the ACL 2026 Adaptive Retriever of Knowledge algorithm to dynamically adjust breadth-depth graph traversal with Voting Rank Fusion (VRF).

This report comprehensively details the system architecture, core algorithms, and end-to-end integration workflows of the HLG LLM Utils framework.

Keywords: Graph RAG, Knowledge Representation, Hierarchical Leiden Algorithm, Cypher Self-Correction, Multi-Agent Systems, LangGraph, FastMCP.

Contents

Executive Abstract	i
1 Introduction & Problem Statement	1
1.1 Background & Motivation	1
1.2 Limitations of Naive Flat RAG	1
1.3 Research Objectives & Core Contributions	1
2 Theoretical Foundations & Related Work	3
2.1 Knowledge Representation Paradigms	3
2.1.1 From Semantic Networks to Property Knowledge Graphs	3
2.2 Vector Space Embeddings & Hybrid Retrieval	3
2.3 Community Detection & The Leiden Algorithm	3
2.4 Microsoft GraphRAG & ARK Exploration	3
3 Formal Knowledge Representation & Schema Design	4
3.1 Property Knowledge Graph Formalization	4
3.2 Neo4j Graph Schema Specification	4
3.3 Triplet Extraction Taxonomy & Claims/Gleaning Pipeline	5
3.3.1 Claims/Covariates Extraction	5
3.3.2 Gleaning (Iterative Extraction Refinement)	5
4 System Architecture & Monorepo Design	6
4.1 UV Workspace Monorepo Architecture	6
4.2 Backend Services & FastAPI Integration	6
5 Advanced Search & Inference Algorithms	7
5.1 Hierarchical Leiden Community Detection	7
5.2 Cypher Self-Correction State Machine	7
6 Conclusion & Task Division	9
6.1 Conclusion	9
6.2 Student Contribution Matrix	9
References	10

List of Figures

4.1	Package Dependencies in the UV Workspace Architecture	6
5.1	LangGraph State Machine for Self-Correcting Cypher Queries	8

List of Tables

3.1	Vertex Attribute Specifications in Neo4j Schema	4
6.1	Task Division Matrix Between Group Members	9

Chapter 1

Introduction & Problem Statement

1.1 Background & Motivation

In the era of Large Language Models (LLMs), natural language understanding and generation have reached unprecedented levels. However, standalone LLMs remain susceptible to hallucinations and knowledge cutoff constraints because their knowledge is statically encoded within model parameters. Retrieval-Augmented Generation (RAG) emerged as the dominant paradigm to ground LLM responses in external authoritative data sources.

Despite its initial success, Naive RAG—which relies on sliding-window text chunking and vector space cosine similarity—exhibits severe structural limitations when applied to complex enterprise Knowledge Representation tasks.

1.2 Limitations of Naive Flat RAG

Through theoretical analysis and empirical observations, we identify three critical failure modes of Naive RAG:

1. **Structural Context Fragmentation:** Fixed-size chunking (e.g., 1,000 tokens) severs conceptual relationships spanning across different paragraphs or separate documents.
2. **Multi-Hop Reasoning Blindspots:** When answering queries requiring multi-step relational chains ($A \rightarrow B \rightarrow C$), vector similarity retrieves chunks mentioning A or C , but misses the crucial intermediate link B .
3. **Corpus-Wide Global Search Failures:** Vector similarity fails on high-level thematic queries (e.g., "What are the primary overarching themes in this document collection?"), as query embeddings do not align with any single text segment.

1.3 Research Objectives & Core Contributions

To address these limitations, this project designs and implements **HLG LLM Utils**, an advanced Graph RAG framework combining Property Knowledge Graphs (PKGs) with multi-agent coordination and hierarchical community detection.

Our primary contributions include:

- Formalizing Property Knowledge Graph schemas $G = (V, E, \mathcal{L}_V, \mathcal{R}_E, \Phi_V, \Phi_E)$ in Neo4j, including entity, relationship, and claims/covariates extraction with iterative Gleaning refinement.
- Implementing Hierarchical Leiden Community Detection for corpus-wide Map-Reduce Global Search.

- Building a LangGraph Cypher Query Self-Correction State Machine with automatic error recovery.
- Integrating the ACL 2026 ARK (Adaptive Retriever of Knowledge) multi-agent trajectory exploration engine.
- Engineering a modular monorepo architecture (`uv workspaces`) decoupling core utilities, vector stores, graph pipelines, and MCP interfaces.

Chapter 2

Theoretical Foundations & Related Work

2.1 Knowledge Representation Paradigms

Knowledge Representation (KR) investigates formalisms for encoding real-world domain knowledge into computational structures capable of reasoning.

2.1.1 From Semantic Networks to Property Knowledge Graphs

The evolution of KR spans Semantic Networks, Minsky’s Frame Systems, Description Logics, and Semantic Web standards (RDF/OWL). Property Knowledge Graphs (PKG) have emerged as the industrial standard, allowing key-value properties to be attached directly to vertices and directed edges.

2.2 Vector Space Embeddings & Hybrid Retrieval

Dense vector embeddings map text segments into continuous vector spaces $\mathbb{R}^d$. To maximize retrieval precision, our system employs Reciprocal Rank Fusion (RRF) to combine lexical BM25 scores with dense vector and graph scores:

$$RRF(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)} \quad (2.1)$$

where M denotes retrieval modalities, $r_m(d)$ is the document rank, and $k = 60$ is the smoothing constant.

2.3 Community Detection & The Leiden Algorithm

Partitioning complex networks into cohesive modules enables macro-level context extraction. The Leiden algorithm guarantees well-connected communities by introducing a local sub-community refinement step missing in Louvain. The modularity quality function $\mathcal{H}$ is defined as:

$$\mathcal{H} = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \gamma \frac{k_i k_j}{2m} \right) \delta(\sigma_i, \sigma_j) \quad (2.2)$$

where A_{ij} is the adjacency matrix, k_i, k_j are node degrees, m is the total edge weight, γ is the resolution parameter, and $\delta(\sigma_i, \sigma_j) = 1$ when nodes share a community.

2.4 Microsoft GraphRAG & ARK Exploration

Recent work by Microsoft Research [1] demonstrated the efficacy of hierarchical community summarization for global queries. Furthermore, the ARK framework [2] established autonomous agentic graph exploration balancing breadth (`global_search`) and depth (`neighborhood_exploration`).

Chapter 3

Formal Knowledge Representation & Schema Design

3.1 Property Knowledge Graph Formalization

We formally model the Property Knowledge Graph $\mathcal{G}$ as a 6-tuple:

$$\mathcal{G} = (V, E, \mathcal{L}_V, \mathcal{R}_E, \Phi_V, \Phi_E) \quad (3.1)$$

where V is the set of entity vertices, $E \subseteq V \times \mathcal{R}_E \times V$ is the set of directed typed edges, $\mathcal{L}_V = \{\text{Document, Chunk, Entity, Community, Claim}\}$ represents node labels, $\mathcal{R}_E = \{\text{HAS_CHUNK, MENTIONS, ...}\}$ contains relationship types, and Φ_V, Φ_E map property key-value attributes onto vertices and edges.

3.2 Neo4j Graph Schema Specification

Table 3.1: Vertex Attribute Specifications in Neo4j Schema

Node Label ($\mathcal{L}_V$)	Property Keys	Technical Description & Data Types
:Document	id, filename, status	Source document identifier, file name, ingestion status (String).
:Chunk	id, text, index, embedding	Text segment, raw content, sequence index, vector embedding $\mathbb{R}^d$.
:Entity	id, type, description, embedding	Uppercase entity ID, category type, summary description, vector embedding $\mathbb{R}^d$.
:Community	id, level, title, summary, findings	Synthetic community ID, hierarchy level l , title, LLM summary, JSON findings.
:Claim	id, subject_id, claim_type, claim_status, description	Knowledge claim (Covariate): type (FACTUAL, TEMPORAL, CAUSAL, QUANTITATIVE, STATUS_CHANGE), status (STATED, INFERRED, DISPUTED), text, date (String).

The graph edges enforce structural integrity:

$$(d : \text{Document}) \xrightarrow{\text{HAS_CHUNK}} (c : \text{Chunk}) \quad (3.2)$$

$$(c : \text{Chunk}) \xrightarrow{\text{MENTIONS}} (e : \text{Entity}) \quad (3.3)$$

$$(e_1 : \text{Entity}) \xrightarrow{\text{RELATED_TO}\{\text{description, weight}\}} (e_2 : \text{Entity}) \quad (3.4)$$

$$(e : \text{Entity}) \xrightarrow{\text{BELONGS_TO}\{\text{level}\}} (c : \text{Community}) \quad (3.5)$$

$$(c_1 : \text{Community}) \xrightarrow{\text{PARENT_OF}} (c_2 : \text{Community}) \quad (3.6)$$

$$(e : \text{Entity}) \xrightarrow{\text{HAS_CLAIM}} (cl : \text{Claim}) \quad (3.7)$$

$$(cl : \text{Claim}) \xrightarrow{\text{REFERENCES}} (e : \text{Entity}) \quad (3.8)$$

3.3 Triplet Extraction Taxonomy & Claims/Gleaning Pipeline

The system formalizes knowledge extraction from unstructured text via LLM structured output (Pydantic schemas):

$$T_{\text{extract}}(C_k) \rightarrow (\{e_i = (\text{name}_i, \text{type}_i, \text{desc}_i)\}, \{r_{ij} = (e_i, \text{rel_type}, e_j, \text{desc}_{ij}, w_{ij})\}) \quad (3.9)$$

All entity names are uppercased and trimmed: $e_{\text{upper}} = \text{UPPER}(\text{TRIM}(\text{name}_i))$.

3.3.1 Claims/Covariates Extraction

Beyond entities and relationships, the system extracts **Claims** – factual assertions anchored to source entities. Each claim is classified along two dimensions:

- **Claim Type:** FACTUAL, TEMPORAL, CAUSAL, QUANTITATIVE, STATUS_CHANGE.
- **Claim Status:** STATED (explicit in text), INFERRED (implied), DISPUTED (contradicted).

Claims are persisted as `:Claim` nodes in Neo4j, linked to their subject entity via `HAS_CLAIM` and to referenced entities via `REFERENCES`. SHA-256 hashing on $(\text{subject_id}, \text{claim_description})$ ensures cross-document deduplication.

3.3.2 Gleaning (Iterative Extraction Refinement)

To maximize knowledge coverage, the system implements **Gleaning** as recommended by Microsoft GraphRAG [1]. After initial extraction, the LLM is prompted to re-read the source text alongside the already-extracted entities and relationships, identifying any missed knowledge:

1. Construct a gleaning prompt containing the list of previously extracted entities and relationships.
2. The LLM returns **only new** entities and relationships not already present.
3. The system deduplicates by key (id_{upper}) for entities and $(\text{source}, \text{target}, \text{type})$ for relationships.
4. Repeat for up to R rounds (`max_gleaning_rounds`), with early stopping when no new knowledge is found.

Chapter 4

System Architecture & Monorepo Design

4.1 UV Workspace Monorepo Architecture

The **HLG LLM Utils** codebase is organized as a unified uv workspace across 9 decoupled packages:

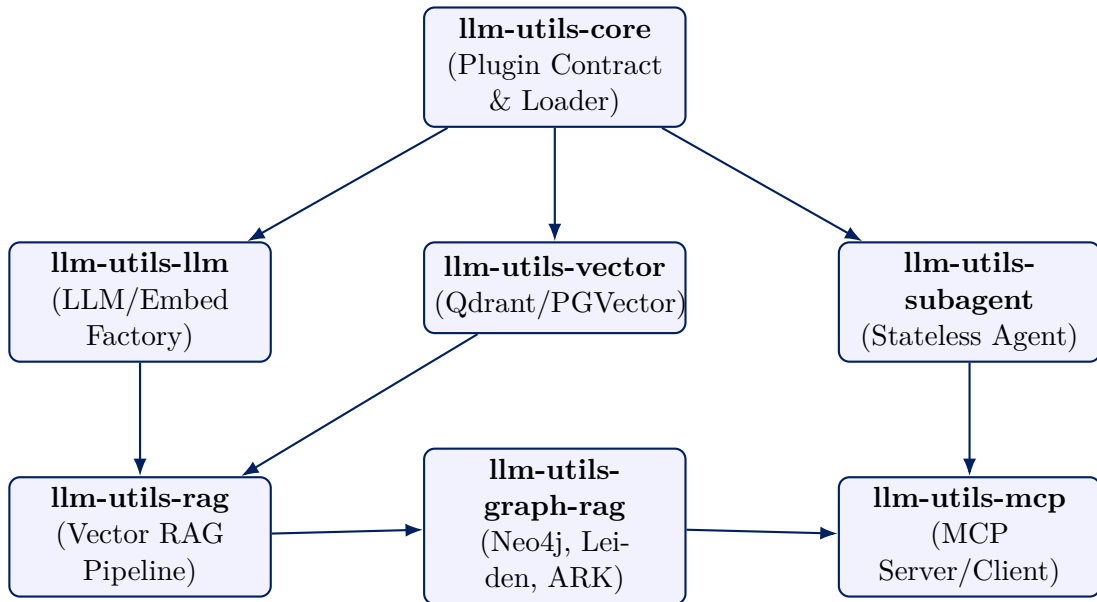


Figure 4.1: Package Dependencies in the UV Workspace Architecture

4.2 Backend Services & FastAPI Integration

The Backend (BE/app/) implements a thread-safe Lazy Singleton registry (`services/registry.py`) providing instances for `GraphQueryService`, `CommunityDetectionService`, and `GlobalSearchService`. During initialization, the registry explicitly connects the Neo4j Graph Store and establishes a fulltext search index (`ensure_fulltext_index`) to support BM25 keyword queries directly on the knowledge graph. WebSocket streaming endpoints (`/ws/chat`) leverage LangGraph `astream_events` to deliver real-time token and tool execution streams.

Chapter 5

Advanced Search & Inference Algorithms

5.1 Hierarchical Leiden Community Detection

The Hierarchical Leiden Algorithm partitions entity graph G across levels $l \in [0, L_{\max} - 1]$:

Algorithm 1: Hierarchical Leiden Partitioning and Summarization

Input: Entity Graph $G = (V, E, W)$, Base resolution γ_0 , Max levels $L_{\max}$

Output: Hierarchical Community Tree C_{tree} summarized by LLM

```
1:  $G_{current} \leftarrow G$ 
2: for  $l = 0$  to  $L_{\max} - 1$  do
3:    $\gamma_l \leftarrow \gamma_0 \times (0.5)^l$ 
4:    $P_l \leftarrow \text{FindPartitionLeiden}(G_{current}, \gamma_l)$ 
5:    $P_{refined} \leftarrow \text{RefineSubCommunities}(G_{current}, P_l)$ 
6:   foreach Community  $c \in P_{refined}$  do
7:      $SubGraph \leftarrow \text{ExtractEntitiesAndEdges}(G, c)$ 
8:      $Report \leftarrow \text{LLM\_Summarize}(SubGraph)$ 
9:      $\text{SaveToNeo4j}(c, l, Report.title, Report.summary, Report.findings)$ 
10:   $G_{current} \leftarrow \text{AggregateGraph}(G_{current}, P_{refined})$ 
```

5.2 Cypher Self-Correction State Machine

A LangGraph state machine translates natural language to Cypher with an automated 3-iteration error recovery loop:

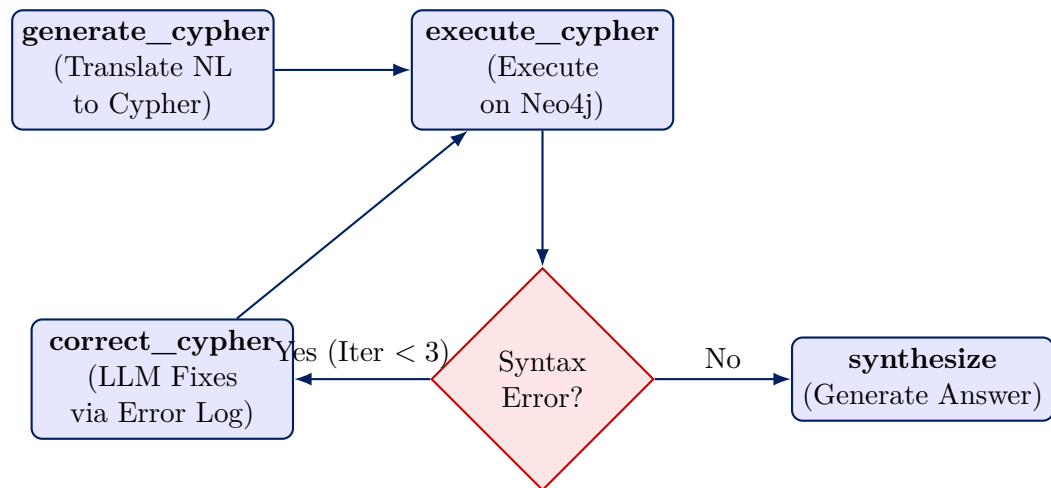


Figure 5.1: LangGraph State Machine for Self-Correcting Cypher Queries

Chapter 6

Conclusion & Task Division

6.1 Conclusion

The **HLG LLM Utils** framework successfully addresses the shortcomings of Naive RAG by combining Property Knowledge Graphs, Hierarchical Leiden Community Detection, Cypher Query Self-Correction, and ARK multi-agent graph traversal into a cohesive, production-ready architecture.

6.2 Student Contribution Matrix

Table 6.1: Task Division Matrix Between Group Members			
Student Name	ID	Specific Contributions & Key Responsibilities	Contribution
Nguyen Hoai Nam	25C01014	- Designed Monorepo uv workspace architecture (llm-utils-core, llm-utils-graph-rag). - Implemented Neo4j Graph Store and Hierarchical Leiden Community Detection (community.py). - Built Map-Reduce Global Search engine (global_search.py). - Authored academic LaTeX report documentation.	50%
Pham Cung Le Nhan Vu	25C01049	- Built LangGraph Cypher Query Self-Correction state graph (workflow/graph.py). - Implemented ARK Autonomous Multi-Agent Traversal framework (ark_agent.py, ark_retriever.py). - Engineered FastAPI backend and Qdrant / PGVector integrations. - Optimized retrieval performance and finalized system integration.	50%

Bibliography

- [1] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [2] Michael Polonuer, Soumya Sanyal, Xiang Ren, and Kai-Wei Chang. Autonomous knowledge graph exploration with adaptive breadth-depth retrieval. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pages 714–730, 2026.